

Voglio scrivere un piccolo linguaggio alla lisp in typescript, scrivendomi la mia libreria di parser combinator, poi un interprete per ambienti web, infine un compilatore che emetta delle istruzioni per isa riscv, anche questo emulato in ambiente web, e che disponga di un semplice sistema operativo multiprogrammato.

Thought for 10s >

Ottimo progetto — ambizioso e molto divertente. Ti do un piano pratico passo-passo + codice d'esempio (TypeScript) per cominciare subito: una piccola libreria di **parser combinator**, un interprete Lisp minimale per il web, una pipeline di compilazione (AST → IR → semplici istruzioni stile RISC-V) + uno **scheletro di emulatore RISC-V** in JS/TS e infine un'idea di **sistema operativo multiprogrammato** da far girare sull'emulatore.

Seguo questo ordine: obiettivi → milestone → API e codice esempio (parser combinator + parser Lisp + interprete) → design del compilatore e mappa istruzioni → emulatore RISC-V minimale + OS multiprogrammazione → strumenti, testing e prossimi passi.

Obiettivi e milestone

1. Milestone 1 — Parser combinator

- Implementare parser combinator in TS (primitives + combinators principali).
- Testare con un parser per la sintassi Lisp (numero, simbolo, lista, quote).

2. Milestone 2 — Interprete Lisp nel browser

- AST + ambiente lexically scoped (chiusure), special forms: `quote`, `if`, `define`, `lambda`, `begin`, `set!`, `apply` / `call`.
- REPL web (textarea + console) + funzioni native di base (math, I/O console).

3. Milestone 3 — Compilatore → IR → RISC-V subset

- Tradurre AST in IR semplice (stack/accumulator o SSA molto semplice).
- Sax down a RISC-V-like instruction set (RV32I subset: `add`, `sub`, `lw`, `sw`, `jal`, `beq`).
- Semplice spilling su stack + calling convention minimale.

4. Milestone 4 — Emulatore RISC-V in browser

- Emulare registri, memoria, fetch/decode/execute per subset di istruzioni.
- UI per vedere registri/memoria/PC.

5. Milestone 5 — OS multiprogrammato minimale

- Implementare process table, context switch (round-robin), system calls per

I/O/exit/yield, timer interrupt simulato.

- Caricare più "binaries" (programmi compilati dal tuo linguaggio) e farli convivere.

Architettura (alta livello)

- Front-end: parser combinator → AST.
- Interprete (per sviluppo rapido/REPL).
- Compilatore: AST → IR → Backend (RISC-V subset).
- Emulator runtime: esegue istruzioni RISC-V.
- OS layer sopra l'emulatore: scheduler, syscalls, memoria virtuale/isolata semplificata.

Parser combinator (TypeScript) — API + implementazione essenziale

Ti do una libreria piccola ma completa di base. Puoi estenderla.

ts

 Copy code

```
// parser.ts
export type Input = { src: string; pos: number };
export type Result<T> = { ok: true; value: T; rest: Input } | { ok: false;
export type Parser<T> = (input: Input) => Result<T>;

export const input = (s: string): Input => ({ src: s, pos: 0 });

function slice(inp: Input, n: number) {
  return { src: inp.src, pos: inp.pos + n };
}

export const char = (c: string): Parser<string> => inp => {
  if (inp.src[inp.pos] === c) return { ok: true, value: c, rest: slice(inp,
  return { ok: false, err: `expected '${c}'`, pos: inp.pos };
};

export const regex = (re: RegExp): Parser<string> => inp => {
  const m = re.exec(inp.src.slice(inp.pos));
  if (m && m.index === 0) {
    return { ok: true, value: m[0], rest: slice(inp, m[0].length) };
  }
  return { ok: false, err: `regex ${re} didn't match`, pos: inp.pos };
};

export const str = (s: string): Parser<string> => inp => {
  if (inp.src.slice(inp.pos, inp.pos + s.length) === s)
    return { ok: true, value: s, rest: slice(inp, s.length) };
  return { ok: false, err: `string "${s}" not found`, pos: inp.pos };
};
```

```

    return { ok: false, err: expected "${s}" , pos: inp.pos };
};

export const map = <A,B>(p: Parser<A>, f: (a: A)=>B): Parser<B> => inp => {
    const r = p(inp);
    if (!r.ok) return r;
    return { ok: true, value: f(r.value), rest: r.rest };
};

export const seq = <A,B>(pa: Parser<A>, pb: Parser<B>): Parser<[A,B]> => inp => {
    const ra = pa(inp);
    if (!ra.ok) return ra;
    const rb = pb(ra.rest);
    if (!rb.ok) return rb;
    return { ok: true, value: [ra.value, rb.value], rest: rb.rest };
};

export const alt = <A>(...ps: Parser<A>[]): Parser<A> => inp => {
    for (const p of ps) {
        const r = p(inp);
        if (r.ok) return r;
    }
    return { ok: false, err: `no alternatives matched`, pos: inp.pos };
};

export const many = <A>(p: Parser<A>): Parser<A[]> => inp => {
    let cur = inp;
    const out: A[] = [];
    while (true) {
        const r = p(cur);
        if (!r.ok) break;
        out.push(r.value);
        cur = r.rest;
        // avoid infinite loop if p can accept empty
        if (cur.pos === inp.pos) break;
    }
    return { ok: true, value: out, rest: cur };
};

export const many1 = <A>(p: Parser<A>) => map(seq(p, many(p)), ([h,t]) => |

export const opt = <A>(p: Parser<A>): Parser<A | null> => inp => {
    const r = p(inp);
    if (r.ok) return r;
    return { ok: true, value: null, rest: inp };
};

export const ws = regex(/^\\s*/);

```

```
export const between = <A,B,C>(open: Parser<B>, p: Parser<A>, close: Parser<B>): Parser<A> => inp => thunk() =>
  map(seq(seq(open, p), close), ([_,a],_) => a);

export const lazy = <A>(thunk: ()=>Parser<A>): Parser<A> => inp => thunk(inp);
```

Questo è sufficiente per scrivere il parser Lisp.

Parser per Lisp (sintassi minimale)

Grammar: `expr ::= number | symbol | list | quote`, `list ::= '(' expr* ')'`

ts

 Copy code

```
// lisp_parser.ts
import * as P from './parser';

type Atom = { type: 'number' | 'symbol', value: string };
export type Expr = Atom | List | Quote;
export type List = { type: 'list', items: Expr[] };
export type Quote = { type: 'quote', value: Expr };

const numberP = P.map(P.regex(/^[-+]?[0-9]+(\.[0-9]+)?/), s => ({ type: 'number', value: s }));
const symbolP = P.map(P.regex(/^([^\s()'\[\]]+)/), s => ({ type: 'symbol', value: s }));

const lparen = P.map(P.seq(P.ws, P.char('(')), _ => null);
const rparen = P.map(P.seq(P.ws, P.char(')')), _ => null);
const quoteCh = P.map(P.seq(P.ws, P.char('\''')), _ => null);

export const expr: P.Parser<Expr> = P.lazy(() => P.map(
  P.seq(P.ws, P.alt(numberP, symbolP, listP, quotedP)),
  ([_, e]) => e
));

export const listP: P.Parser<List> = P.map(
  P.between(P.seq(P.ws, P.char('(')), P.many(expr), P.seq(P.ws, P.char(')'))),
  items => ({ type: 'list', items })
);

export const quotedP: P.Parser<Quote> = P.map(
  P.seq(quoteCh, expr),
  ([_, e]) => ({ type: 'quote', value: e })
);

export function parseAll(s: string): Expr[] {
  let inp = P.input(s);
  const out: Expr[] = [];
  while (true) {
    const r = expr(inp);
    out.push(r);
    inp = P.input(s.slice(r.value.length));
  }
}
```

```

        if (!r.ok) break;
        out.push(r.value);
        inp = r.rest;
        // avoid infinite loop
        if (inp.pos >= inp.src.length) break;
    }
    return out;
}

```

Interprete Lisp minimal (environment, eval)

Dare un interprete semplice con supporto a `lambda` con chiusura, `define`, `if`, `quote`, chiamate normali.

ts

 Copy code

```

// lisp_interp.ts
import { Expr } from './lisp_parser';

type Env = { parent?: Env, map: Map<string, any> };

function env(parent?: Env): Env { return { parent, map: new Map() }; }
function envGet(e: Env, k: string): any {
    if (e.map.has(k)) return e.map.get(k);
    if (e.parent) return envGet(e.parent, k);
    throw new Error(`Unbound symbol ${k}`);
}
function envSet(e: Env, k: string, v: any) { e.map.set(k, v); }

type LispFunc = (...args: any[]) => any;
type Closure = { params: string[], body: Expr[], env: Env };

function makeGlobalEnv(): Env {
    const g = env();
    envSet(g, '+', (...a: number[]) => a.reduce((x, y) => x + y, 0));
    envSet(g, '-', (a: number, b?: number) => b === undefined ? -a : a - b);
    envSet(g, '*', (...a: number[]) => a.reduce((x, y) => x * y, 1));
    envSet(g, '/', (a: number, b: number) => a / b);
    envSet(g, 'print', (...args: any[]) => { console.log(...args); return null; });
    return g;
}

export function evalExpr(e: Expr, envCur: Env): any {
    if (e.type === 'number') return Number(e.value);
    if (e.type === 'symbol') return envGet(envCur, e.value);
    if (e.type === 'quote') return e.value;
    if (e.type === 'list') {
        if (e.items.length === 0) return null;

```

```

const first = e.items[0];
if (first.type === 'symbol') {
  const sym = first.value;
  // special forms
  if (sym === 'define') {
    const name = (e.items[1] as any).value;
    const val = evalExpr(e.items[2], envCur);
    envSet(envCur, name, val);
    return val;
  }
  if (sym === 'lambda') {
    const params = (e.items[1] as any).items.map((p:any)=>p.value);
    const body = e.items.slice(2);
    return { params, body, env: envCur } as Closure;
  }
  if (sym === 'if') {
    const cond = evalExpr(e.items[1], envCur);
    if (cond) return evalExpr(e.items[2], envCur);
    return evalExpr(e.items[3], envCur);
  }
  if (sym === 'begin') {
    let res = null;
    for (let i=1;i<e.items.length;i++) res = evalExpr(e.items[i], envCur);
    return res;
  }
}
// function application
const fn = evalExpr(e.items[0], envCur);
const args = e.items.slice(1).map(a=>evalExpr(a, envCur));
// closure?
if ((fn as Closure).params) {
  const cl = fn as Closure;
  const newEnv = env(cl.env);
  for (let i=0;i<cl.params.length;i++) newEnv.map.set(cl.params[i], args[i]);
  let res: any = null;
  for (const b of cl.body) res = evalExpr(b, newEnv);
  return res;
}
if (typeof fn === 'function') return (fn as LispFunc)(...args);
throw new Error("Attempt to call non-function");
}
}

```

Questi snippet ti danno un REPL funzionante molto rapidamente: scrivi un parser che produce AST, poi `evalExpr` su ogni espressione in un `makeGlobalEnv()`.

Compilatore: AST → IR → RISC-V (idea)

1. **IR:** istruzioni semplici tipo `CONST rX, imm`, `LOAD_ARG rX, k`, `CALL rRes, fnAddr`, `RET rX`, `ADD rX, rY, rZ`, `STORE rX, [SP+offset]` ecc. Scegli un IR che faciliti il lowering (stack machine o three-address).
2. **Lowering a RISC-V:** mappa IR su istruzioni RV32I:
 - `ADD r1, r2, r3` → `add x1, x2, x3`
 - `CONST` → `addi` o `lui` + `addi` per grandi costanti
 - `CALL` → `jal` / `jalr` con calling convention minimale (usiamo `x1 = ra`, `x2..x7` per args)
 - Gestione stack: `sw` / `lw` con `sp` (`x2`).
3. **Calling convention minimale:**
 - `sp` holds stack pointer
 - `ra` stored on call
 - arguments in registers `a0..a7` (map to `x10..x17`)
 - return in `a0` (`x10`)
4. **Register allocation:** per cominciare usa una strategia semplice:
 - Stack-based lowering: `espr` → push constants on stack, then use `lw` / `sw` for temps (semplice ma sub-ottimale).
 - Poi implementa un allocator greedy (assign virtual regs to physical regs, spill when needed).

Esempio di lowering semplice (pseudo)

LISP call (+ 1 2)

- AST → IR:

```
sql
```

 Copy code

```
PUSH 1
```

```
PUSH 2
```

```
CALL builtin:+ 2 -> result on stack
```

- IR → RISC-V:

```
nginx
```

 Copy code

```
addi sp, sp, -8
```

```
sw x10, 0(sp)          # push 1 into mem (if value in a0)
```

```
sw x11, 4(sp)          # push 2
```

```
li a0, 1               # load immediate 1 into a0 (arg1)
```

```
li a1, 2               # arg2
```

```
jal ra, builtin_add    # call builtin routine
```

```
addi sp, sp, 8
```

I builtin puoi implementarle nell'emulatore come routine native (syscalls) oppure come funzioni in memoria compilate.

Emulatore RISC-V minimale (TOY) - skeleton

Emulatore RISC-V minimale (TS) — skeleton

Implementa fetch/decode/execute ciclo in TS per subset: add, sub, addi, lw, sw, beq, bne, jal, jalr, lui.

ts

 Copy code

```
// riscv_emulator.ts
export class CPU {
  regs: Int32Array;
  pc: number;
  mem: Uint8Array;

  constructor(memSize = 64*1024) {
    this.regs = new Int32Array(32);
    this.pc = 0;
    this.mem = new Uint8Array(memSize);
  }

  readWord(addr: number) {
    return (this.mem[addr] | (this.mem[addr+1]<<8) | (this.mem[addr+2]<<16) |
    (this.mem[addr+3]<<24));
  }

  writeWord(addr:number, val:number) {
    this.mem[addr] = val & 0xFF;
    this.mem[addr+1] = (val>>8)&0xFF;
    this.mem[addr+2] = (val>>16)&0xFF;
    this.mem[addr+3] = (val>>24)&0xFF;
  }

  step() {
    const instr = this.readWord(this.pc);
    // decode minimal set (opcodes simplified)
    const opcode = instr & 0x7f;
    if (opcode === 0x33) { // R-type (add, sub)
      const funct3 = (instr>>12)&0x7;
      const funct7 = (instr>>25)&0x7f;
      const rd = (instr>>7)&0x1f;
      const rs1 = (instr>>15)&0x1f;
      const rs2 = (instr>>20)&0x1f;
      if (funct3===0 && funct7===0x00) this.regs[rd] = (this.regs[rs1] + tl
      if (funct3===0 && funct7===0x20) this.regs[rd] = (this.regs[rs1] - tl
      this.pc += 4;
      return;
    }
    // implement addi, lw, sw, beq, jal, jalr as needed...
    throw new Error(`Unknown opcode ${opcode.toString(16)} at pc ${this.pc}`);
  }
}
```

```

run(maxSteps=100000) {
  for (let i=0;i<maxSteps;i++) this.step();
}
}

```

Estendi con implementazioni di `addi` (opcode 0x13), `lw/sw` (0x03 / 0x23), `beq` (0x63), `jal` (0x6f), `jalr` (0x67).

Sistema operativo minimale (design)

- **Process:** contiene `pid`, `registers snapshot`, `pc`, `stack pointer`, `memory region` (offset in emulated mem).
- **Scheduler:** round-robin con quantum (numero di istruzioni per processo). Implementa in JS con loop che esegue N istruzioni del processo corrente, poi salva contesto (`regs+pc`) e carica quello del prossimo.
- **Syscalls:** implementa tabella syscall: `print(int fd, addr, len)`, `exit(code)`, `sleep(ms)` (emulata), `yield()` (cambia processo).
- **Interruzione timer:** nel browser usa `setInterval` per chiamare scheduler superficialmente, oppure fai scheduler cooperativo basato su chiamata `yield` nel codice compilato.

UI/Dev experience nel web

- Editor (CodeMirror/Monaco) per scrivere sorgenti Lisp.
- Pulsanti: Parse, Interpret, Compile, Run in emulator, Spawn process.
- Visualizzazione: registers, memory hexdump, process list, console log.
- Hot reload: compila e carica in memoria dell'emulatore.

Testing e debugging

- Unit test per parser combinator (mocha/jest).
- Property testing per parse/unparse (generare espressioni e verificare roundtrip).
- Emulatore: test istruzioni singole con fixture (carica programmino, step e verifica registri).
- For debugging, crea una modalità "trace" che stampa ogni istruzione eseguita e i registri.

Performance e ottimizzazioni future

- Migrare hot paths in WebAssembly (se vuoi spedire codice bench).
- Migliorare register allocator: linear scan o graph coloring per codice più efficiente.
- Aggiungere garbage collection (se il linguaggio ha heap e alloc dinamica) o usare memoria gestita via OS del runtime.
- Implementare TOC (Tail Call Optimization) sia in interpretato che in compilato.

- Implementare TCO (Tail Call Optimization) sia in interprete che in compilatore.

Piccoli consigli pratici

- Lavora a piccoli passi: prima REPL interpretato, poi compila funzioni semplici, poi chiamate, poi closures.
- Mantieni il formato binario del "program" semplice: se carichi funzioni compilate nell'emulatore, definisci un piccolo header per entrypoints e fornisci una tabella di simboli per debugger.
- Per il RISC-V, inizia con un subset minimo utile: add, addi, lw, sw, jal, jalr, beq, bne, lui. Aggiungi altre istruzioni solo se servono.
- Metti tutto su Git (branch per ogni milestone). Commit piccoli.

Esempi di roadmap giornaliera

- Giorno 1: parser combinator + parser Lisp + REPL interpretato (operativo).
- Giorno 2-3: arricchire interprete con lambda, closure, environment persistente; implementa `map`, `filter` builtin.
- Giorno 4-6: design IR + tradurre piccole funzioni → test con codice aritmetico.
- Giorno 7-10: backend RISC-V mapping + emulator skeleton + test per alcune istruzioni.
- Giorno 11-14: implementa scheduler semplice e syscalls; crea UI minimale per lanciare due processi.

Se vuoi, posso:

- generare un repository starter completo con i file di cui sopra (package.json, build scripts, esbuild config, esempio REPL in browser) — te lo scrivo qui come struttura e codice pronto da copiare;
- oppure partire subito con il **compilatore IR → RISC-V**: ti do un esempio concreto che prende espressioni lambda semplici e genera istruzioni RV32I per una calling convention minima;
- o creare una pagina web demo con REPL + emulatore UI (HTML+TS).

Dimmi quale di questi preferisci che faccia ora (repo starter / es. compila → RISC-V / demo web REPL + emulator) e procedo subito con il codice completo e pronto da eseguire.



Cosa basta implementare nel mio compilatore per generare degli ELF eseguibili su sistemi linux sopra RISC-V, oltre che ad avere un runtime "bare metal"

caricabile da disco, con tecniche BIOS semplici?

Domanda eccellente — e molto “da ingegnere del linguaggio”: arrivare a generare **binari ELF eseguibili veri su Linux RISC-V**, e in parallelo saper generare un **bare-metal loader** caricabile “da BIOS” (o meglio da firmware), è esattamente il punto di contatto tra compilatore, sistema operativo e architettura.

Vediamo in dettaglio e in ordine tecnico crescente cosa devi implementare.

Obiettivo

Hai due target distinti:

1. Target Linux RISC-V

- genera un file **ELF eseguibile conforme** (ELF64 o ELF32 a seconda del sistema, in genere ELF64-LP64).
- linkabile o caricato da Linux (`ld.so` , `execve`), con `_start` come entry point.

2. Target Bare-metal (BIOS-like)

- genera un’immagine binaria grezza (es. `.bin` o `.img`), che può essere caricata in RAM all’indirizzo noto (`0x80000000`) e saltata via `PC = entry` .
- contiene un piccolo **runtime** (setup stack, zero BSS, entry C-like, chiamata `main`).

— ELF su Linux: cosa devi implementare

A. Struttura minima di un file ELF a 64-bit

Un ELF eseguibile minimale ha:

```
pgsql  Copy code

[ ELF header ]
[ Program header table ]
[ Segmenti: .text, .data, .bss, .rodata ]
```

Non serve la sezione symbol table o relazioni se non usi il linker dinamico.

ELF Header (64-bit)

Campo	Descrizione
<code>e_ident[EI_MAG0..3]</code>	<code>0x7F 'E' 'L' 'F'</code>
<code>e_ident[EI_CLASS]</code>	<code>2</code> → <code>ELFCLASS64</code>
<code>e_ident[EI_DATA]</code>	<code>1</code> → <code>little endian</code>

1. Riceve i registri secondo ABI RISC-V (a0=argc, a1=argv, a2=envp).
2. Chiama `main(argc, argv, envp)`.
3. Esegue la syscall `exit()` con il valore di ritorno.

Codice assembly tipico:

asm

Copy code

```
.global _start
_start:
    mv a0, a0    # argc
    mv a1, a1    # argv
    mv a2, a2    # envp
    call main
    mv a0, a0
    li a7, 93    # syscall exit
    ecall
```

D. Syscall ABI

Linux RISC-V usa:

- a7 = syscall number
- a0 .. a5 = args
- a0 = return
- ecall = trap to kernel.

Ad esempio:

syscall	number	args
write	64	a0=fd, a1=buf, a2=len
exit	93	a0=status

2 — Bare-metal / BIOS-like runtime

In questo caso, nessun ELF loader: generi un'immagine **raw binaria** (oppure un ELF "flat" con un loader tuo).

A. Layout della memoria

arduino

Copy code

```
0x80000000 → entry point
[ boot stub: init stack, clear BSS, call main ]
[ codice main ]
[ dati ]
```

B. Boot stub assembly

asm

Copy code

```
.section .text
.globl _start
_start:
    la sp, _stack_top
    call main
halt:
    j halt

.bss
.space 4096
_stack_top:
```

C. Runtime C-like (o TS-compiled)

Devi implementare:

- Init stack pointer
- Zero BSS
- Setup global pointers se serve (gp , tp)
- Call main
- Loop infinito o trap on exit.

D. Output binario

Dopo la compilazione, "appiattisci" l'ELF rimuovendo header e caricando .text e .data consecutivamente:

bash

Copy code

```
riscv64-unknown-elf-objcopy -O binary prog.elf prog.bin
```

Oppure scrivi direttamente un writer TS che produce il file:

css

Copy code

```
[ codice machine ]
[ dati inizializzati ]
```

E. Boot (BIOS-like)

Puoi caricare con QEMU:

perl

Copy code

```
qemu-system-riscv64 -machine virt -kernel prog.bin
```

Oppure usare `-bios none` per simulare un boot diretto senza firmware.

3 — Cosa serve *realmente* nel tuo compilatore

Livello	Cosa devi generare
Assembler output	Codice RISC-V conforme all'ABI (RV64I base, sezione <code>.text</code> , <code>.data</code> , etichette, simboli).
ELF writer	Costruisci ELF header e program header <code>PT_LOAD</code> , e serializza le sezioni nei corretti offset.
Symbol resolver	Mappa simboli a indirizzi virtuali durante l'emissione.
Relocation (base)	Se supporti label forward reference, risolvi displacement nei branch/jal.
Runtime	Fornisci <code>_start</code> , implementazione minimale di <code>ecall</code> wrapper, e funzioni di base (<code>write</code> , <code>exit</code>).
Bare metal runtime	Aggiungi <code>_start</code> assembly + <code>main</code> C/TS compiled code + data init.
ABI compliance	Rispetta RISC-V calling convention: <code>a0-a7</code> argomenti, <code>ra</code> return address, <code>sp</code> 16-byte aligned.

4 — Suggerimento pratico per te

Nel tuo caso (compilatore scritto in TypeScript):

1. Genera codice **assembly testuale RISC-V** (`.s`) → poi assemblalo con `riscv64-linux-gnu-as + ld → ELF`.
2. Poi implementa un **writer ELF** in TypeScript (solo header + `PT_LOAD`).
3. Alla fine aggiungi un "linker integrato" per generare ELF statici direttamente dal compilatore.

Per il bare metal:

- Genera immagine raw (`.bin`).
- Scrivi piccolo boot stub assembler.
- Carica in QEMU con `-kernel o -bios none`.

5 — Toolchain utile per sperimentare

Duoi testare entrambi i target con:

Fuori testare entrambi i target con:

bash

Copy code

```
# Linux ELF target
riscv64-linux-gnu-gcc -nostdlib -static test.S -o test.elf
qemu-riscv64 ./test.elf

# Bare metal
riscv64-unknown-elf-as -o test.o test.S
riscv64-unknown-elf-ld -Ttext=0x80000000 -o test.elf test.o
riscv64-unknown-elf-objcopy -O binary test.elf test.bin
qemu-system-riscv64 -machine virt -kernel test.bin -nographic
```

In breve: checklist per ELF RISC-V

Requisito	Descrizione
ELF Header	Tipo ET_EXEC , machine EM_RISCV
PH Table	almeno due PT_LOAD segmenti (.text, .data)
Entry Point	_start valido
Sezioni	.text, .data, .bss allineate a 0x1000
Runtime	_start + syscall exit , write
ABI	RV64I calling convention
Facoltativo	symbol table, rel section (per debug)

Se vuoi, posso scriverti **un ELF writer completo in TypeScript** (circa 150 righe) che prenda un buffer di codice .text e .data e produca un ELF RISC-V64 minimale eseguibile da Linux o QEMU.

Vuoi che te lo mostri?



Considerando che il mio linguaggio sarà garbage collected con lifetime di oggetti limitata allo scope, che algoritmi semplici posso scegliere per fare il

Ottima domanda — e molto concreta, visto che il tuo linguaggio Lisp-like ha *scopi di vita limitati allo scope*, quindi puoi permetterti qualcosa di estremamente più semplice rispetto a un garbage collector full-featured come mark-and-sweep o generational.

Vediamo passo per passo che algoritmi o tecniche potresti usare per il “**pruning**” **degli alberi di riferimenti**, cioè la liberazione delle strutture dati alla fine dello scope.

Premesse

Hai detto:

- GC con lifetime **limitato allo scope** (quindi *non serve* gestire oggetti che sopravvivono a un blocco);
- target iniziale: **interprete o compilatore con runtime leggero**, anche “bare metal”.

In questo scenario, puoi trattare la memoria come **una serie di regioni (arena)** o come **alberi isolati** da eliminare in blocco.

1. Region-based allocation (a.k.a. arena allocation)

È *la più semplice* e più coerente col tuo modello.

Idea

Ogni scope (funzione, blocco, closure temporanea) ha una propria *arena* o *region*. Gli oggetti vengono allocati nella region corrente e, quando esci dallo scope, liberi l'intera region con un colpo solo.

ts

[Copy code](#)

```
class Region {
  objects: any[] = []
  alloc<T>(obj: T): T {
    this.objects.push(obj)
    return obj
  }
  free() {
    this.objects.length = 0
  }
}

function withRegion<R>(fn: (r: Region) => R): R {
```

```

const region = new Region()
const result = fn(region)
region.free()
return result
}

```

♦ Vantaggi

- Nessun tracciamento di riferimenti.
- Zero overhead in tempo di esecuzione.
- Nessun GC vero e proprio — solo un “reset”.

♦ Svantaggi

- Non puoi avere riferimenti che escono dallo scope.
- Tutti gli oggetti della regione vengono distrutti assieme.

È perfetto se il linguaggio lo *impone semanticamente*: le closure non possono catturare dati vivi oltre lo scope.

2. Reference trees + depth-based pruning

Se il linguaggio costruisce esplicitamente alberi (come in un Lisp, dove tutto è una lista o cons), puoi rappresentare le strutture con riferimenti padre-figlio e distruggere l'albero *ricorsivamente* quando nessuno lo referencia.

Algoritmo semplice

- Ogni nodo ha un contatore dei riferimenti diretti.
- Quando un riferimento a un nodo viene rimosso:
 - decrementa il contatore,
 - se diventa 0 → distruggi ricorsivamente i figli.

ts

[Copy code](#)

```

class Node {
  refCount = 1
  children: Node[] = []
  retain() { this.refCount++ }
  release() {
    if (--this.refCount === 0) {
      for (const c of this.children) c.release()
    }
  }
}

```

♦ Vantaggi

- Nessun ciclo? Funziona perfettamente.

- Implementazione molto intuitiva.

♦ Svantaggi

- Non gestisce cicli (ma puoi evitarli facilmente in un Lisp-like ben definito).
- Ogni incremento/decremento ha un costo minimo (ma costante).

🧠 3. "Stack GC" per linguaggi puramente funzionali

Se tutto ciò che sopravvive vive *sullo stack* (nessun heap persistente), puoi:

- mantenere un piccolo heap temporaneo per gli oggetti creati durante l'esecuzione di una funzione;
- quando torni dalla funzione, liberi tutto.

È una versione *ancora più rigida* delle regioni, usata ad esempio nei Lisp interpretati primitivi (come l'early Scheme di Steele & Sussman).

⚙️ 4. Variante ibrida

Puoi combinare:

- **Reference counting** per oggetti condivisi o persistenti;
- **Region free** per gli scope locali.

Esempio:

- ogni `let` introduce una nuova *region*;
- gli oggetti che escono dal `let` vengono copiati o "promossi" nella region del padre, incrementando il refcount.

💡 Riassunto pratico

Modello	Complessità	Cicli	Overhead	Uso tipico
Region-based	♦ Minima	✗	♦ $O(1)$ free	Scope rigidi, linguaggi stack-based
Ref-counting	♦ Media	⚠️ Problema con cicli	♦ $O(n)$	Strutture condivise
Mark & Sweep	▲ Alta	✓	▲ Elevato	Heap persistente, dinamico
Copying GC	▲ Alta	✓	▲ Elevato	VM con heap giovane/vecchio

🔧 Suggerimento per il tuo caso

Dato che il tuo linguaggio:

- è Lisp-like (quindi strutture ad albero);
- è *scope-lifetime-based*;
- mira a un **ISA RISC-V** e a un runtime leggero,

👉 **Inizia con una region-based allocation con eventuale refcount per i cons condivisi.**

Puoi anche aggiungere un comando di debug tipo `(gc)` che stampa le regioni attive e la loro dimensione.

Se vuoi, posso mostrarti un piccolo **diagramma di flusso** di come un "region-based GC" si integra con l'interprete Lisp (allocazione → valutazione → distruzione). Vuoi che te lo disegni in ASCII o come immagine?



Da tutto quello che ti ho detto, che cosa è necessario per avere una runtime per riscv base, con garbage collection, funzioni e oggetti allocabili dinamicamente?

Domanda perfetta — e molto concreta per chi vuole passare da *interprete web* a *runtime RISC-V nativo*.

Ti rispondo in modo **architetturale**, cioè elencandoti **tutti i sottosistemi minimi** che servono per avere un **runtime Lisp-like con GC, funzioni e heap dinamico**, in grado di girare su una macchina RISC-V (sia Linux che bare-metal).

🧱 1. Sottosistemi fondamentali del runtime

Per un linguaggio con *allocazione dinamica* e *funzioni di ordine superiore*, ti servono **cinque livelli fondamentali**:

Livello	Ruolo	Note per RISC-V
A. Memory Manager	astrazione base di allocazione, free, regioni, heap	interfaccia con <code>sbrk</code> , <code>mmap</code> , o un <i>bump allocator</i>

B. Garbage Collector	gestione automatica della memoria	anche minimale (regioni o refcount)
C. Object Model	rappresentazione uniforme di cons, liste, closure, numeri, simboli	tag sugli oggetti o punteggio di tipo (tagged pointers)
D. Function/Closure Runtime	gestione degli ambienti lessicali e chiamate	frame dinamici nello stack o heap; supporto per funzioni anonime
E. OS/Platform Interface	I/O, timer, file, syscall minime	se bare-metal → device driver base; se Linux → syscall wrapper

2. Memory manager

Obiettivi

- Fornire `alloc(size)` e `free(ptr)`
- Gestire uno *heap* contiguo (ad esempio con un bump allocator)

Variante 1 — Linux

Puoi usare direttamente:

c

[Copy code](#)

```
void* alloc(size_t n) { return sbrk(n); }
void free(void* p) {} // no-op se usi GC
```

Variante 2 — Bare metal

Devi avere:

- un simbolo `_end` (fine del bss)
- un puntatore `heap_ptr` che cresce
- un limite massimo (`HEAP_TOP`)

c

[Copy code](#)

```
extern char _end;
char* heap_ptr = &_amp_end;
void* alloc(size_t n) {
    void* p = heap_ptr;
    heap_ptr += n;
    return p;
}
```

3. Garbage collector

Dato che vuoi qualcosa di semplice:

- **Region-based** o **refcounting** inizialmente.
- Poi puoi aggiungere un *mark & sweep* quando vuoi permettere oggetti che sopravvivono a più scope.

Mark & Sweep base (solo heap lineare)

1. Mantieni una lista degli oggetti allocati.
2. Parti da *root references* (registri, stack, globali).
3. Marca tutto ciò che è raggiungibile.
4. Fai sweep liberando ciò che non è marcato.

Per performance, implementi `alloc()` come un *freelist allocator* che riusa blocchi liberi.

4. Object model

Devi definire una **rappresentazione comune** per ogni tipo dinamico.

Struttura tipica

c

Copy code

```
typedef enum { OBJ_INT, OBJ_PAIR, OBJ_FUNC } ObjType;

typedef struct Obj {
    ObjType type;
    uint8_t marked;
    struct Obj* next; // per il GC
    union {
        struct { struct Obj* car; struct Obj* cdr; } pair;
        struct { int64_t value; } integer;
        struct { void* code; struct Obj* env; } func;
    };
} Obj;
```

Tutti gli oggetti stanno sullo heap; l'interprete manipola solo puntatori a `Obj`.

5. Function runtime

Ogni funzione Lisp-like ha:

- un **environment frame** (tabella variabili → valori)

- un **codice** (bytecode o AST)
- la possibilità di chiudere su un *environment padre* (closure)

Esecuzione

- Quando chiami una funzione, crei un *activation record* nello stack.
- Quando crei una closure, allochi un oggetto `OBJ_FUNC` con:
 - un puntatore al codice (o AST)
 - un riferimento all'ambiente catturato

Per implementare tutto in un emulatore RISC-V:

- lo stack è un semplice array o area riservata;
- puoi simulare le chiamate dinamiche con un piccolo *call dispatcher*:

c

Copy code

```
void call(Obj* fn, Obj* args) {
    push_frame(fn->func.env);
    interpret(fn->func.code, args);
    pop_frame();
}
```

6. Interface con il sistema (syscalls / BIOS)

Dipende dal contesto:

♦ Se sei su Linux (user mode)

Puoi emettere syscalls RISC-V:

asm

Copy code

```
li a7, 64      # write
ecall
```

Quindi puoi avere I/O base, memoria, file ecc.

♦ Se sei bare metal

Serve un piccolo *bios layer* con:

- driver per console (UART)
- timer o scheduler per task multipli
- inizializzazione heap e stack
- eventualmente una `syscall_table` software per interfacciare task user-space

7. Multiprogrammazione

Per un "micro-OS" in user-space o bare metal:

- Scheduler cooperativo o round-robin
- Context switching (salvi registri, stack pointer)
- Ogni processo ha un heap separato (region allocator)
- GC per ogni processo indipendente, o globale con *safepoint*

Esempio di pseudo-codice per cooperativo:

c

Copy code

```
void yield() {  
    save_context(current);  
    current = next_thread();  
    load_context(current);  
}
```

8. Compilatore → ELF

Quando emetti codice per RISC-V:

- genera sezione `.text` con codice
- `.data` per stringhe e costanti
- `.bss` per heap e variabili globali
- header ELF corretto (e.g. `e_machine = EM_RISCV`)

Poi puoi:

- farlo eseguire su Linux RISC-V (qemu-user)
- oppure caricarlo come *bare-metal binary* tramite QEMU `virt` e il tuo "bios".

Riassunto architetturale finale

Componente	Descrizione	Livello
Allocatore	bump / freelist / region	♦ base
GC	region-based o mark-sweep	♦ memoria
Object model	tagged union per valori	♦ semantica
Closure engine	ambiente + codice	♦ linguaggio
Scheduler	opzionale (per OS)	⚙ runtime
ELF linker	output nativo	📦 compilatore
RISC-V user	UO e memoria	🏠 piattaforma

Se vuoi, posso disegnarti uno schema **a blocchi** del runtime RISC-V del tuo linguaggio (da heap a GC, da closure a interprete, fino a OS minimale).
Vuoi che te lo mostri in ASCII o come immagine?



Che syntactic sugar potrei implementare nell'assembly per il mio riscv?

Bellissima domanda, Valerio — questa è la parte *davvero divertente* 😊
Scrivere un linguaggio assembly "con syntactic sugar" significa trasformare un **ISA spoglio come RISC-V** in un piccolo linguaggio espressivo, leggibile e produttivo — come se fosse una via di mezzo tra *macroassembler*, *Forth* e *mini-C*.

Vediamo insieme:

1. cosa **non c'è nativamente** nel RISC-V,
2. che **sugar** puoi introdurre senza rompere la semantica,
3. e che **macro-strutture** possono rendere più piacevole scrivere codice anche per il tuo compilatore Lisp-like.

🧩 1. Cosa manca nel RISC-V "puro"

RISC-V è volutamente minimale, quindi mancano:

- pseudoistruzioni complesse (loop, if, call convenzionali),
- gestione automatica dello stack o dei frame,
- operazioni a livello di struttura o array,
- addressing simbolico "ad alto livello",
- etichette locali, macro, condizioni inline, e costanti più potenti.

🧠 2. Syntactic sugar "micro" (sulle istruzioni)

Queste sono estensioni di comodo che puoi implementare nel *tuo assembler* senza alterare il modello di esecuzione:

- ♦ 1. Operazioni immediate automatiche

Permetti costanti anche oltre i 12 bit gestiti da `addi`, facendole tradurre in sequenze

`lui + addi`:

asm

Copy code

```
li x5, 0x12345678 ; sugar
```

→

asm

Copy code

```
lui x5, 0x12345
addi x5, x5, 0x678
```

♦ 2. Etichette relative e locali

Permetti di definire etichette annidate e relative, come in NASM:

asm

Copy code

```
loop:
    addi t0, t0, -1
    bnez t0, .loop
```

Il parser calcola automaticamente la label `".loop"` relativa alla label esterna.

♦ 3. Pseudoistruzioni di confronto

Aggiungi `sugar` come:

asm

Copy code

```
beqz x5, label ; beq x5, x0, label
bnez x5, label ; bne x5, x0, label
bltz x5, label ; blt x5, x0, label
bgtz x5, label ; bgt x5, x0, label
```

e combinazioni con immediati:

asm

Copy code

```
beqi x5, 10, label ; addi t0, x0, 10; beq x5, t0, label
```

♦ 4. Accesso a memoria simbolico

asm

[Copy code](#)

```
lw a0, var
sw a1, [array + 8]
```

→ il compilatore risolve `var` e `array` come indirizzi statici nella `.data`.

◆ 5. Macro per stack frame

Definisci direttive tipo:

asm

[Copy code](#)

```
.proc my_function(nlocals=3)
    push ra
    push s0
    addi sp, sp, -12
    ...
.endproc
```

Oppure sugar per chiamate:

asm

[Copy code](#)

```
call print_int    ; jal ra, print_int
ret               ; jalr x0, ra, 0
```

3. Sugar “meso” (strutturali / linguistici)

Questi rendono il linguaggio più leggibile e simile a un linguaggio ad alto livello:

◆ 1. Blocchi `if`, `while`, `for` espansi dal pre-processore

Puoi introdurre una sintassi tipo C-like, espansa in branch e label:

asm

[Copy code](#)

```
if x5 == 0 {
    call zero_case
} else {
    call nonzero_case
}
```

→

asm

[Copy code](#)

```

beqz x5, .L_else
jal ra, zero_case
j .L_end
.L_else:
jal ra, nonzero_case
.L_end:

```

♦ 2. Pseudo-variabili locali

Suggerisci dichiarazioni simboliche che generano offset nello stack frame:

asm

[Copy code](#)

```

.locals {
    counter: 0(sp)
    ptr: 4(sp)
}
lw t0, counter
sw t1, ptr

```

o ancora meglio:

asm

[Copy code](#)

```

mov counter, 0
while counter < 10 {
    call print_int(counter)
    add counter, counter, 1
}

```

♦ 3. Costrutti Lisp-like

Dato il tuo obiettivo, puoi anche voler introdurre macro “funzionali” in stile Lisp ma compilabili in RISC-V:

asm

[Copy code](#)

```

(def fib (n)
  (if (<= n 1)
      n
      (+ (fib (- n 1)) (fib (- n 2)))))

```

→ espanso in assembly con chiamate ricorsive e frame corretti.

4. Sugar “macro-architetturale”

Quando implementerai il tuo compilatore, puoi introdurre **macro predefinite** che implementano pattern ricorrenti del tuo runtime:

◆ Heap management

asm

[Copy code](#)

```
alloc t0, 16      ; chiama il runtime allocator
free t0
gc_collect
```

◆ Funzioni Lisp

asm

[Copy code](#)

```
apply f, arglist
lambda entry_label, env
```

◆ Threading / yield

asm

[Copy code](#)

```
spawn f
yield
```

Tutte translate in chiamate al tuo microkernel / runtime multiprogrammato.

5. Sugar per debugging e introspezione

Puoi introdurre pseudoinstruzioni non-eseguibili, gestite solo dall'emulatore o runtime:

asm

[Copy code](#)

```
print "x5 =", x5
dumpmem 0x80000000, 64
assert x5 != 0
```

Utile per il tuo **emulatore web RISC-V**, per osservare memoria, heap, GC e stack live.

6. Riassunto per categorie

Categoria

Esempi di sugar

Effetto

Sintassi immediata	<code>li , beqz , la , mv</code>	leggibilità
Strutturale	<code>if {...} , while {...}</code>	flusso di controllo chiaro
Dichiarativo	<code>.locals , .proc , .data</code> symbolica	struttura e scope
Funzionale	<code>lambda , apply , def</code>	ponte verso linguaggio Lisp-like
Sistema	<code>alloc , spawn , yield</code>	runtime e GC integrati
Debug	<code>print , dumpmem , assert</code>	introspezione e tracciamento

Suggerimento finale

Se vuoi che il tuo assembly diventi una sorta di **“meta linguaggio target”** per il tuo Lisp — cioè un *IR leggibile* — il trucco migliore è:

- mantenere un *parser modulare* con **macro espandibili in AST**,
- implementare gli *sugar* come **macro definite in TypeScript**, così il linguaggio evolve da sé.

Posso mostrarti un esempio concreto di come definire il tuo “macroassembler” in TypeScript, con parsing e riscrittura del codice assembly prima dell’emissione delle istruzioni binarie RISC-V.

Vuoi che ti mostri quello schema o un piccolo prototipo di codice?

